

Here's a comprehensive set of **viva questions and answers** for your A* search algorithm implementation, especially tailored for solving problems like the 8-puzzle:

Algorithm Understanding

1. *What is the A algorithm and how does it differ from Dijkstra's algorithm?**

Answer: A* is a best-first search that combines the cost from the start node ($g(n)$) and the estimated cost to the goal ($h(n)$). Dijkstra's algorithm only considers the cost so far ($g(n)$), making A* generally faster and more directed when a good heuristic is used.

2. *What does the $f(n) = g(n) + h(n)$ formula represent in A?**

Answer: It represents the total estimated cost of the cheapest solution through node n . $g(n)$ is the actual cost from the start to n , and $h(n)$ is the estimated cost from n to the goal.

3. *Why is A considered both complete and optimal? Under what conditions?**

Answer: A* is complete and optimal if the heuristic used is **admissible** (never overestimates the cost) and **consistent** (obeys the triangle inequality).

4. *What role does the priority queue (heap) play in A?**

Answer: It ensures that the node with the **lowest f value** is always expanded first. This prioritizes promising paths and leads to efficient goal discovery.

Heuristics

5. **What are admissible and consistent heuristics?**

Answer:

- **Admissible:** Heuristic never overestimates the true cost to the goal.
- **Consistent (Monotonic):** $h(n) \leq \text{cost}(n, n') + h(n')$ for every neighbor n' .

6. Explain the difference between Hamming distance and Manhattan distance.

Answer:

- **Hamming:** Counts number of misplaced tiles.
- **Manhattan:** Sum of vertical and horizontal distances each tile is from its goal position. Manhattan is more accurate for sliding puzzles.

7. What is Linear Conflict, and how does it improve over Manhattan?

Answer: Linear Conflict adds additional penalties when two tiles are in their correct row/column but in the wrong order. It improves accuracy by refining Manhattan distance with conflict awareness.

8. Can a heuristic ever overestimate the cost? What are the consequences if it does?

Answer: If a heuristic overestimates, it's **not admissible**, and A* may return a **non-optimal path**, violating its optimality guarantee.

9. Why did you choose Manhattan distance as your default heuristic in solve()?

Answer: It balances **accuracy and speed**, and it is **admissible and consistent** for sliding puzzles, making it a safe and effective default.

Implementation Specific

10. Why do you use a set for the closed list?

Answer: To enable **fast lookups ($O(1)$)** for already explored states using the node's hash value. It prevents revisiting the same board configuration.

11. Explain how the `__hash__` and `__eq__` methods affect performance and correctness.

Answer: They allow Node objects to be stored and compared in sets. `__hash__` uses the board's tuple form; `__eq__` ensures two nodes with the same board are treated as duplicates.

12. What happens if you don't define the `__lt__` method?

Answer: The priority queue (heapq) won't know how to compare Node objects and will raise a `TypeError` when it tries to sort them.

13. How do you handle revisiting already explored nodes in your open list?

Answer: Before adding a child to the open list, you check if it's already there and if it offers a **lower f cost** than the existing one. If so, you update it.

14. Why do you deep-copy the board in `generateChildNodes()`?

Answer: To ensure child states are **independent** and don't modify the parent state accidentally, preserving tree integrity.

Optimization and Edge Cases

15. What tie-breaking strategy did you implement? Why?

Answer: When f values are equal, nodes with **lower h values** are preferred. This directs the search closer to the goal and reduces unnecessary expansions.

16. *Could your A implementation get stuck in a loop? Why or why not?**

Answer: No, because we track explored states in a **closed set**, preventing revisits and infinite loops.

17. **What would happen if the puzzle is unsolvable? How can you detect that?**

Answer: A* will eventually explore all reachable nodes and return None. Solvability can be pre-checked using **inversion counts** in 8-puzzle.

18. *How can you measure the efficiency of your A algorithm?**

Answer: By tracking:

- Number of **expanded** nodes,
- Number of **explored** (open list) nodes,
- **Time taken** to find the solution,
- **Path length** of the solution.

19. **Is your solution scalable to larger puzzles like 15-puzzle?**

Answer: Not efficiently. Larger puzzles require more memory and time. Optimizations like **Iterative Deepening A*** or **Pattern Databases** may be necessary.

20. **How could you optimize memory usage in your implementation?**

Answer:

- Use **immutable data structures** like tuples instead of lists.
- Use **hash maps** to avoid storing duplicate states.
- Implement **lazy evaluation** or **on-demand state expansion**.

Would you like these as a formatted PDF or slides for your preparation or presentation?